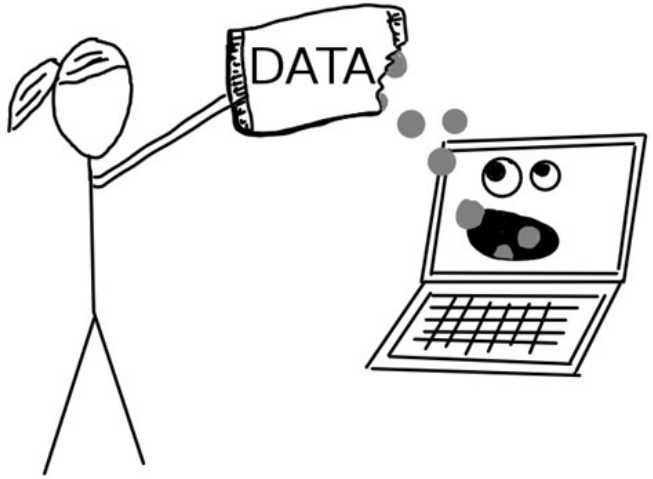
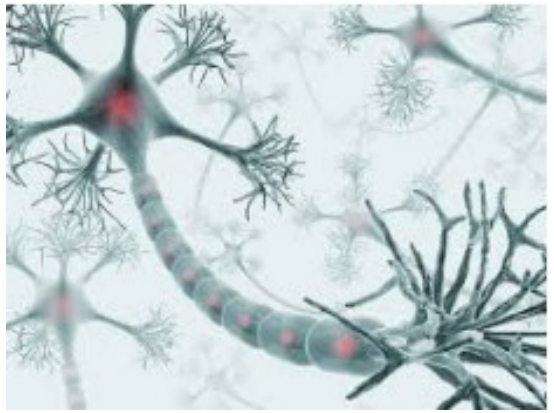




Multi Layer Neural Network

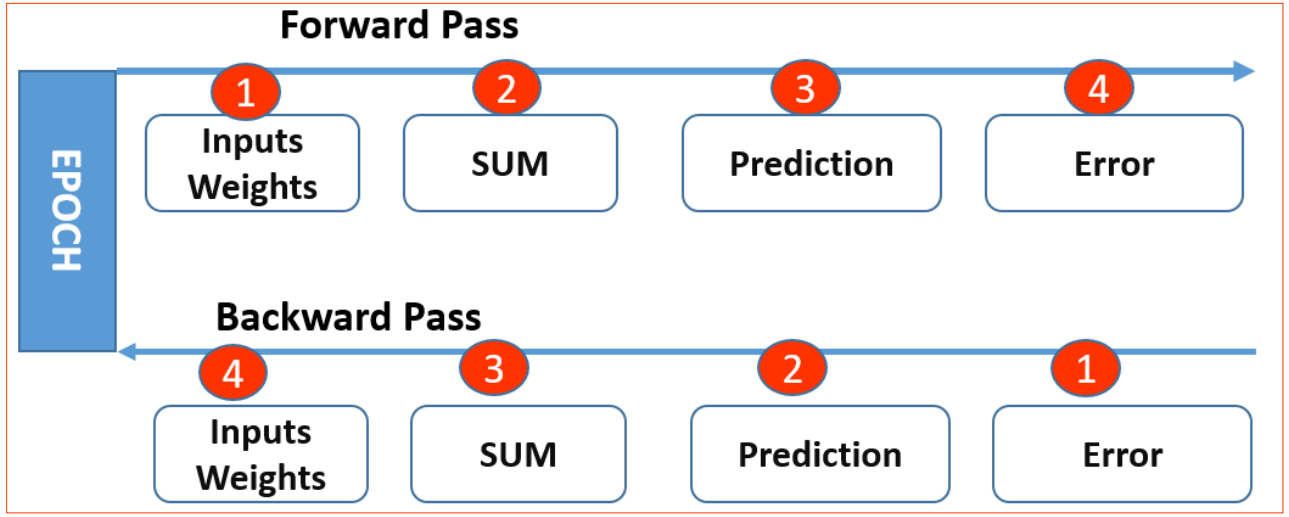
BackPropagation Algorithm

Version 2023

Abdelwahab Naji - Enseignant chercheur
Abdelwahab.naji@gmail.com
Laboratoire Signaux, Systèmes Distribués et Intelligence Artificielle (SSDIA)
ENSET de Mohammedia, Université Hassan II de Casablanca
https://www.youtube.com/c/AbdelwahabNaji

Backpropagation Algorithm

- Après chaque passage en avant (**Forward pass**) à travers un réseau, la rétropropagation effectue un passage en arrière (**Backward pass**) tout en ajustant les paramètres du modèle (poids et biais).
- L'algorithme est utilisé pour entraîner efficacement un réseau de neurones à l'aide d'une méthode appelée **chain rule**



chain rule

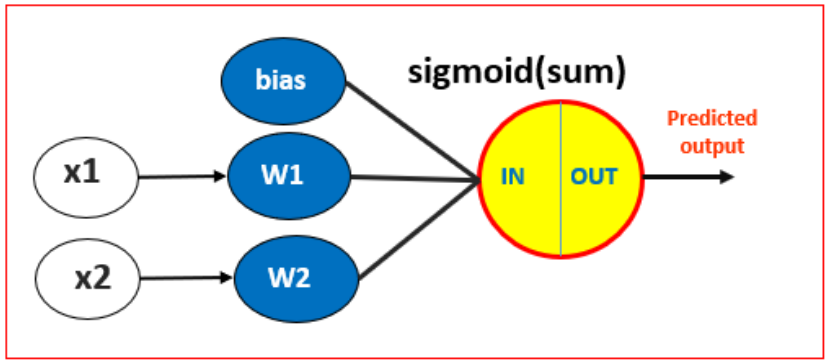
$y = f(u)$

et $u = g(x)$

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

1. Forward pass: pour la prédiction
2. Backward pass: pour la correction

Algorithme



De 1 à n_epochs faire

- Forward pass
- Backward pass

1 **Initialize weights** **Initialize learning rate**

De 1 à n_epochs faire

2

$$sum = \sum_{i=0}^{n} W_i * X_i$$

3 **Activation function →**

$$f(s) = \frac{1}{1+e^{-s}} = output$$

4 **error= desired-**

$$Loss = \frac{1}{2}(desired - predicted)^2$$

5 **Weights update**

$$w_1 = w_1 - learning_rate * \frac{\partial loss}{\partial w_1}$$

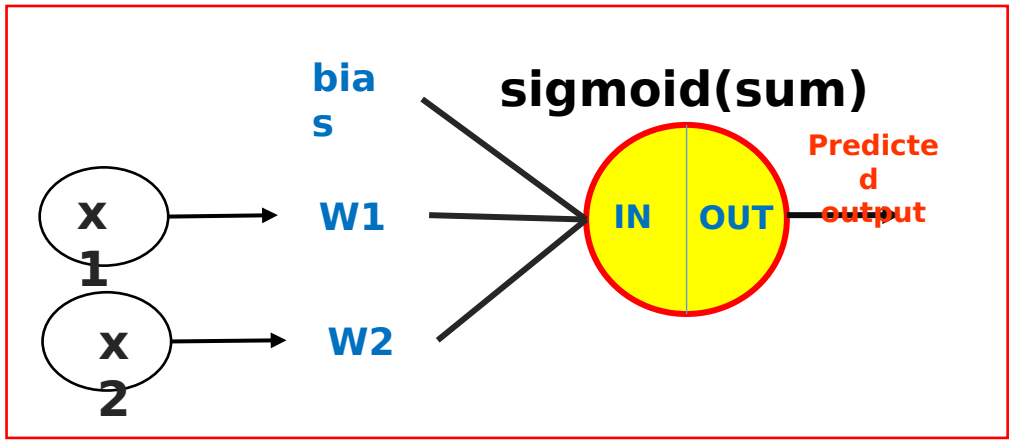
$$w_2 = w_2 - learning_rate * \frac{\partial loss}{\partial w_2}$$

$$bias = bias - learning_rate * \frac{\partial loss}{\partial bias}$$

Exemple

X1	X2	output
0.1	0.3	0.03
…		

learning_rate=0.01



- Initialiser les poids de manière aléatoire
- Mettre à jour les poids en appliquant l’algorithme de backpropabation pour minimiser la fonction Loss
- Tourner l’algorithme en deux itérations

X1	X2	output
0.1	0.3	0.03
…		

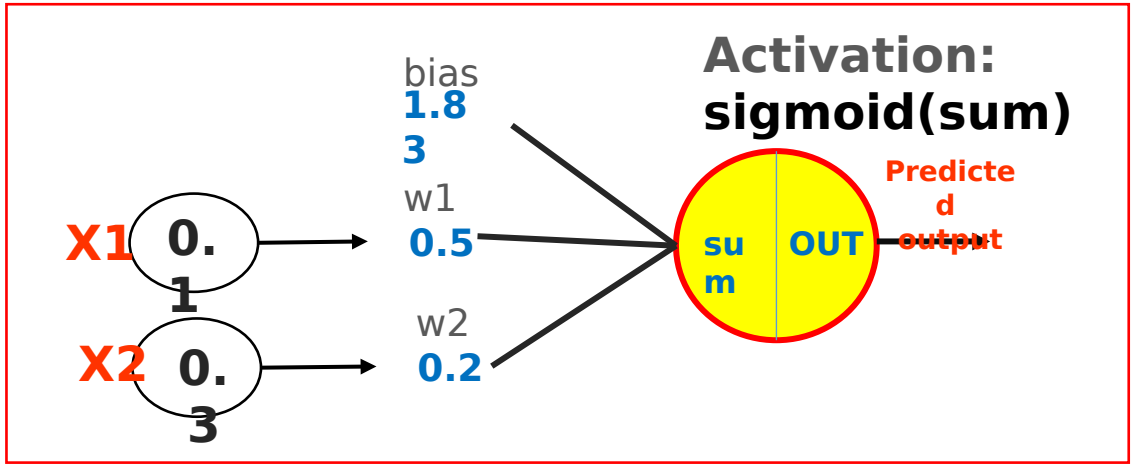
learning_rate= 0.01

Forward pass

Sum=w1*x1+w2*x2+bias

Predicted output=sigmoid(sum)

$$Loss = \frac{1}{2}(desired - predicted)^2$$



Backward pass

$$w_1 = w_1 - learning_rate * \frac{\partial loss}{\partial w_1}$$

$$w_2 = w_2 - learning_rate * \frac{\partial loss}{\partial w_2}$$

$$bias = bias - learning_rate * \frac{\partial loss}{\partial bias}$$

$$\frac{\partial Loss}{\partial w1} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial w1}$$

$$\frac{\partial Loss}{\partial w2} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial w2}$$

$$\frac{\partial Loss}{\partial bias} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial bias}$$

$$\frac{\partial Loss}{\partial predicted} = predicted - desired$$

$$\frac{\partial predicted}{\partial s} = f(s)\,(1 - f(s))$$

$$\frac{\partial s}{\partial w1} = x1$$

$$\frac{\partial s}{\partial w2} = x2$$

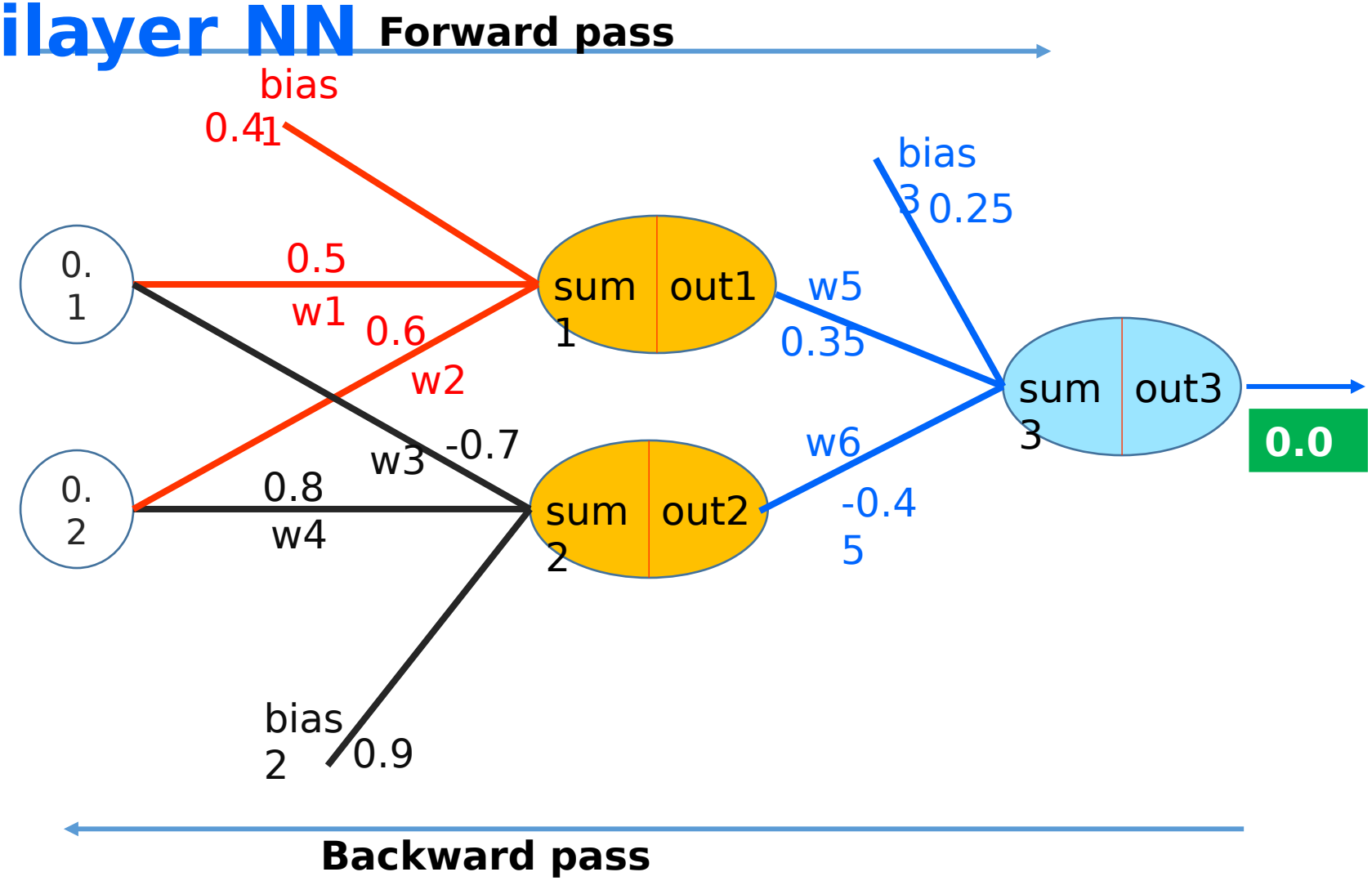
$$\frac{\partial s}{\partial bias} = 1$$

Backpropagation – Algorithm (version1: utiliser des matrices)

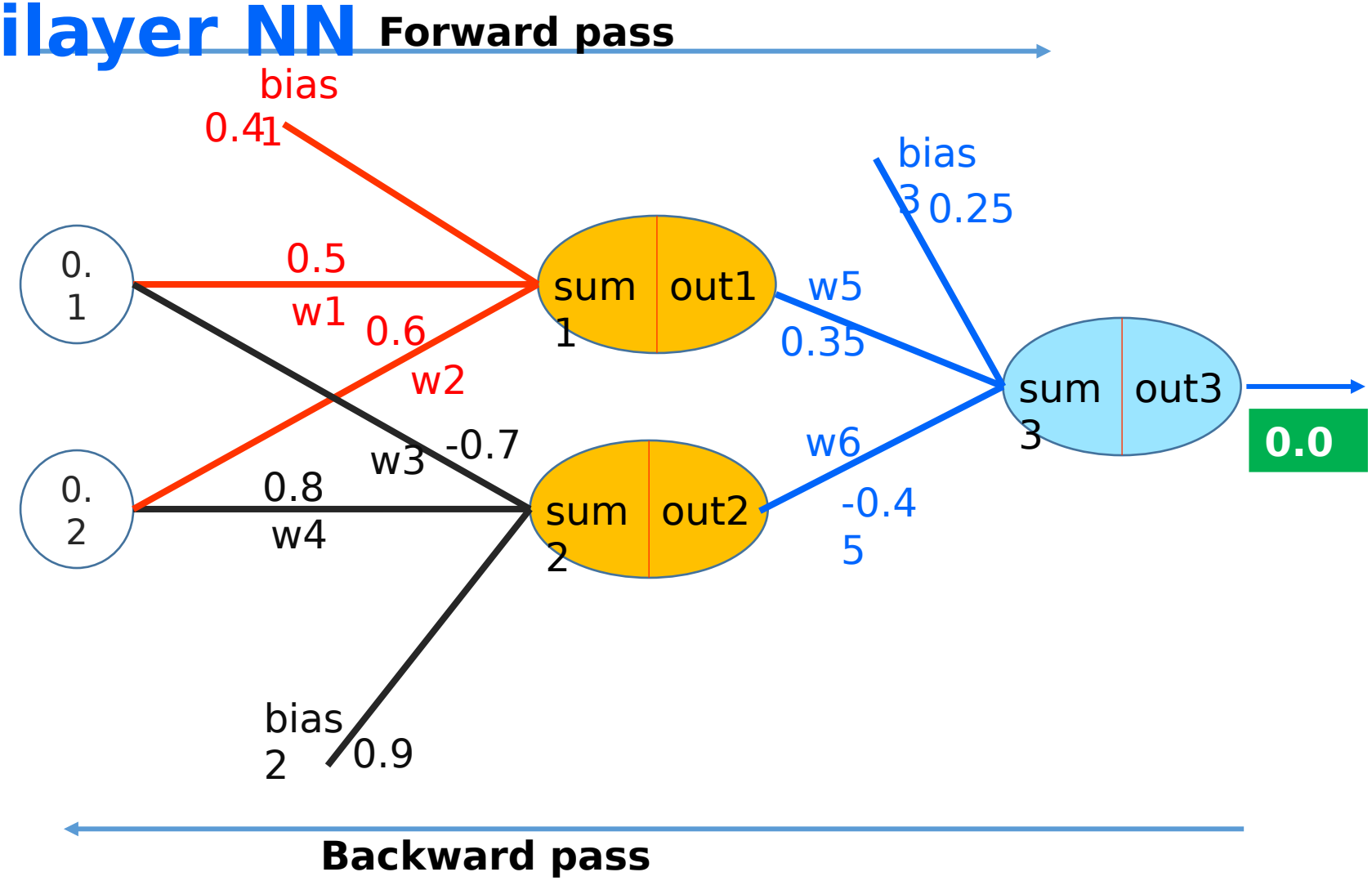
```
Epoch 10000: ----------------------------------
sum:-3.2627453450687973
prediction: 0.036871592334909074
Loss-quadr: 2.360939060859058E-5
0,027   -1,219 -2,900
-----------------------------------------------
```

backpropagation

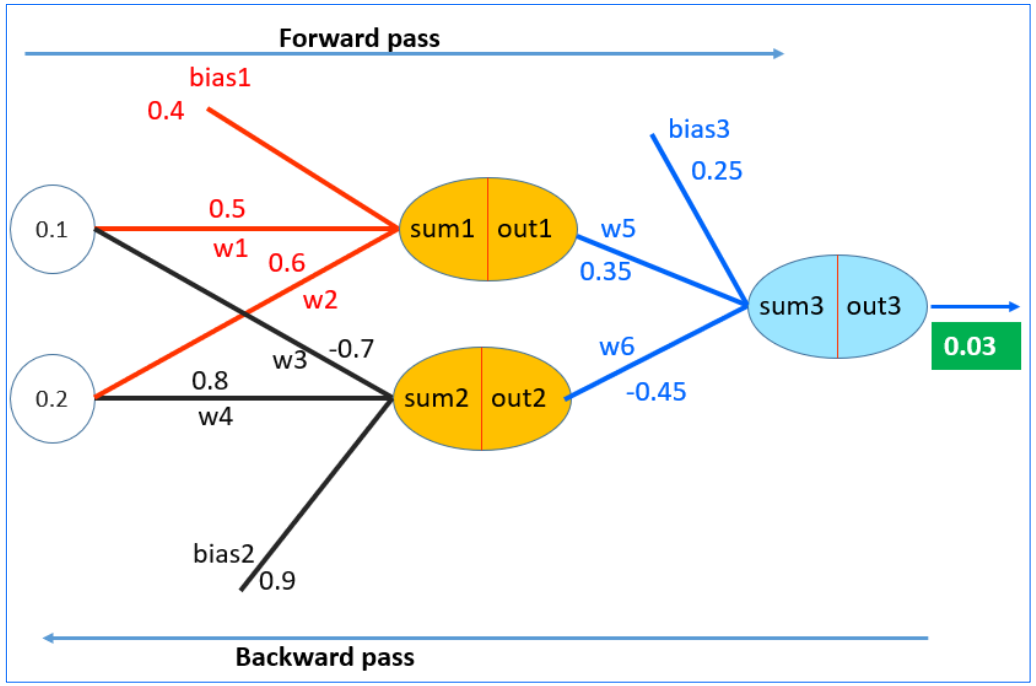
Update weights and bias for Multilayer NN



Update weights and bias for Multilayer NN



Comment?



$$w_1 = w_1 - learning_rate * \frac{\partial loss}{\partial w_1}$$

$$w_2 = w_2 - learning_rate * \frac{\partial loss}{\partial w_2}$$

$$w_3 = w_3 - learning_rate * \frac{\partial loss}{\partial w_3}$$

$$w_4 = w_4 - learning_rate * \frac{\partial loss}{\partial w_4}$$

$$w_{hidden} = w_{hidden} - learning_rate * \frac{\partial loss}{\partial w_{hidden}}$$

$$bias_1 = bias_1 - learning_rate * \frac{\partial loss}{\partial bias_1}$$

$$bias_2 = bias_2 - learning_rate * \frac{\partial loss}{\partial bias_2}$$

$$bias_{hidden} = bias_{hidden} - learning_rate * \frac{\partial loss}{\partial bias_{hidden}}$$

$$w_5 = w_5 - learning_rate * \frac{\partial loss}{\partial w_5}$$

$$w_6 = w_6 - learning_rate * \frac{\partial loss}{\partial w_6}$$

$$w_{out} = w_{out} - learning_rate * \frac{\partial loss}{\partial w_{out}}$$

$$bias_3 = bias_3 - learning_rate * \frac{\partial loss}{\partial bias_3}$$

$$bias_{out} = bias_{out} - learning_rate * \frac{\partial loss}{\partial w_{bias}}$$

1) Update hidden layer weights and bias

Update hidden layer weights and bias

$$w_{hidden} = w_{hidden} - learning_rate * \frac{\partial loss}{\partial w_{hidden}}$$

Calculer $\frac{\partial Loss}{\partial W_hidden}$

$$bias_{hidden} = bias_{hidden} - learning_rate * \frac{\partial loss}{\partial bias_{hidden}}$$

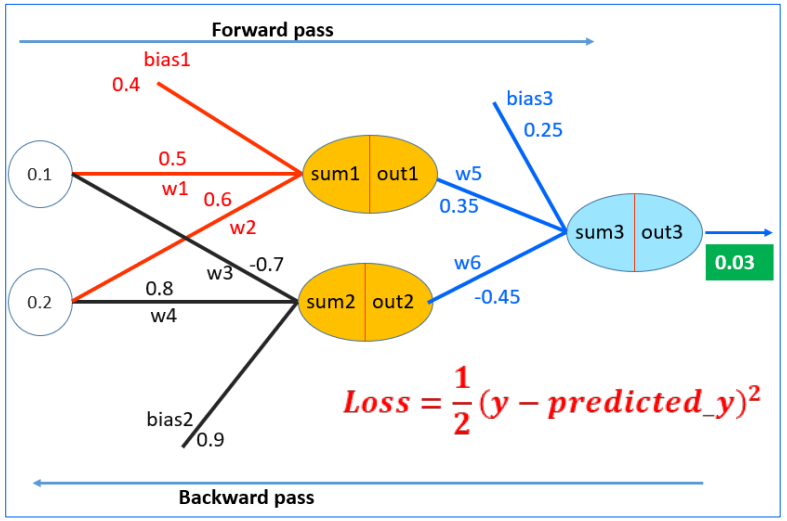
Calculer $\frac{\partial Loss}{\partial bias_hidden}$

Update hidden layer weights

$$\frac{\partial Los}{\partial W_hid}$$

Backpropagation – MultiLayer NN

Utilisation de chain rule pour trouver $\frac{\partial Loss}{\partial W_hidden}$



$$Loss = \frac{1}{2}(y - predicted_y)^2$$

out3=Predicted_y

$$\frac{\partial Loss}{\partial w1} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial out1} * \frac{\partial out1}{\partial sum1} * \frac{\partial sum1}{\partial w1}$$

$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

dsum3_dout1= w5

out1 =sigmoid(sum1)

d_out1_dsum1=sigmoid_derivative(out1)

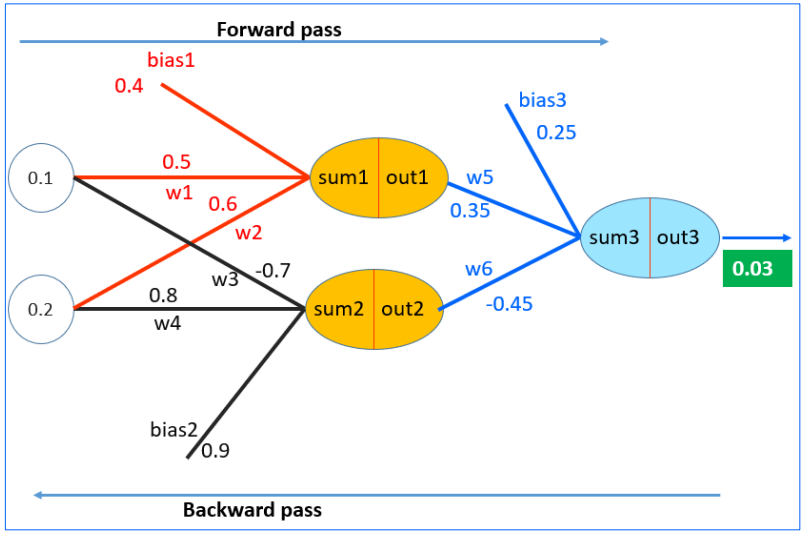
sum1 =w1*x1+w2*x2+bias1

d_sum1_dw1 =x1

dloss_dw1=**(predicted_y-y)* sigmoid_derivative(predicted_y)* w5* sigmoid_derivative(out1)* x1**

Backpropagation – MultiLayer NN

$$Loss = \frac{1}{2}(y - predicted_y)^2$$



$$\frac{\partial Loss}{\partial w2} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial out1} * \frac{\partial out1}{\partial sum1} * \frac{\partial sum1}{\partial w2}$$

$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

dsum3_dout1= w5

out1 =sigmoid(sum1)

d_out1_dsum1=sigmoid_derivative(out1)

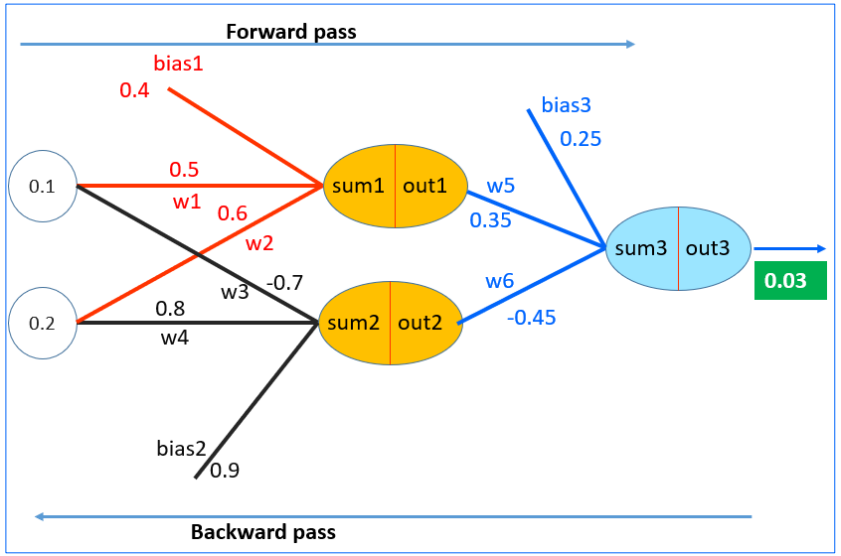
sum1 =w1*x1+w2*x2+bias1

d_sum1_dw2 =x2

dloss_dw2=**(predicted_y-y)* sigmoid_derivative(predicted_y)* w5* sigmoid_derivative(out1)* x2**

Backpropagation – MultiLayer NN

Utilisation de chaine rule pour trouver $\frac{\partial Loss}{\partial W_hidden}$



$$\frac{\partial Loss}{\partial w3} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial out2} * \frac{\partial out2}{\partial sum2} * \frac{\partial sum2}{\partial w3}$$

$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

dsum3_dout2= w6

out2 =sigmoid(sum2)

d_out2_dsum2=sigmoid_derivative(out2)

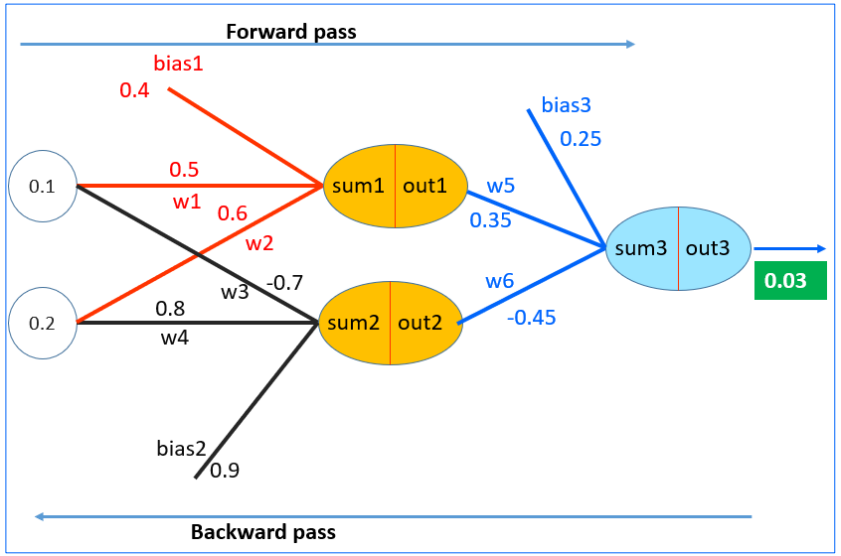
sum2 =w3*x1+w4*x2+bias2

d_sum2_dw3 =x1

dloss_dw3=(predicted_y-y)* sigmoid_derivative(predicted_y)* w6* sigmoid_derivative(out2)* x1

Backpropagation – MultiLayer NN

Utilisation de chaine rule pour trouver $\frac{\partial Loss}{\partial W_hidden}$



$$\frac{\partial Loss}{\partial w4} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial out2} * \frac{\partial out2}{\partial sum2} * \frac{\partial sum2}{\partial w4}$$

$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

dsum3_dout2= w6

out2 =sigmoid(sum2)

d_out2_dsum2=sigmoid_derivative(out2)

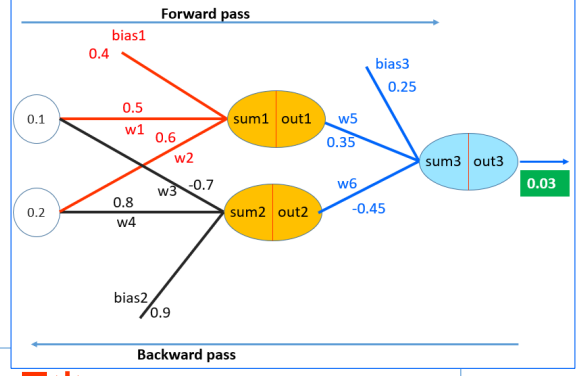
sum2 =w3*x1+w4*x2+bias2

d_sum2_dw4 =x2

dloss_dw4=(predicted_y-y)* sigmoid_derivative(predicted_y)* w6* sigmoid_derivative(out2)* x2

Update hidden layer weights

$$\frac{\partial Loss}{\partial W_hidden}$$



dloss_dw1=(predicted_y-y)* sigmoid_derivative(predicted_y)* w5* sigmoid_derivative(out1)* x1

dloss_dw2=(predicted_y-y)* sigmoid_derivative(predicted_y)* w5* sigmoid_derivative(out1)* x2

dloss_dw3=(predicted_y-y)* sigmoid_derivative(predicted_y)* w6* sigmoid_derivative(out2)* x1

dloss_dw4=(predicted_y-y)* sigmoid_derivative(predicted_y)* w6* sigmoid_derivative(out2)* x2

dloss_dw=(predicted_y-y)* sigmoid_derivative(predicted_y)* output_W* sigmoid_derivative(h_out)* X

Code python

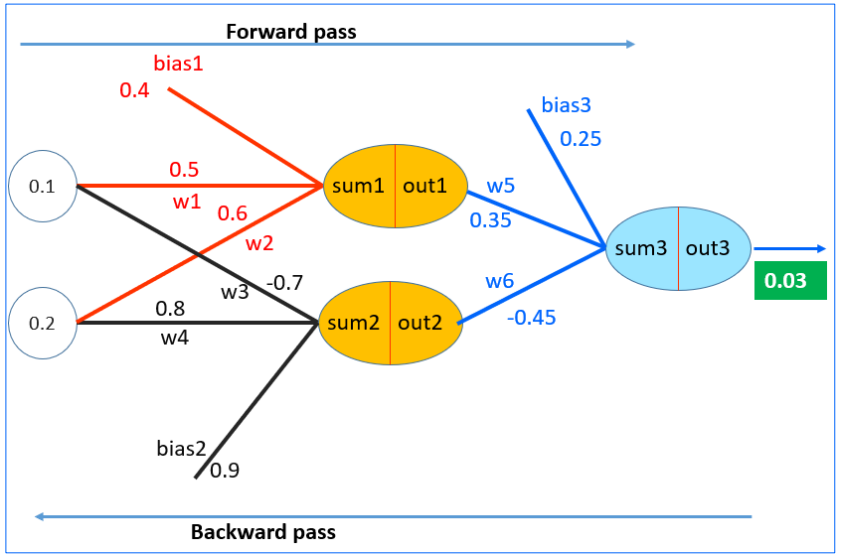
```
dloss_dw_h= X.T.dot((predicted_y-y)*
sigmoid_derivative(predicted_y).dot(output_weights.T)*sigmoid_derivative(hidden_layer_out))
```

Update hidden layer bias

$$\frac{\partial Lo}{\partial bias_h}$$

Backpropagation – MultiLayer NN

Utilisation de chaine rule pour trouver $\frac{\partial Loss}{\partial bias_hidden}$



$$\frac{\partial Loss}{\partial bias1} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial out1} * \frac{\partial out1}{\partial sum1} * \frac{\partial sum1}{\partial bias1}$$

$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

dsum3_dout1= w5

out1 =sigmoid(sum1)

d_out1_dsum1=sigmoid_derivative(out1)

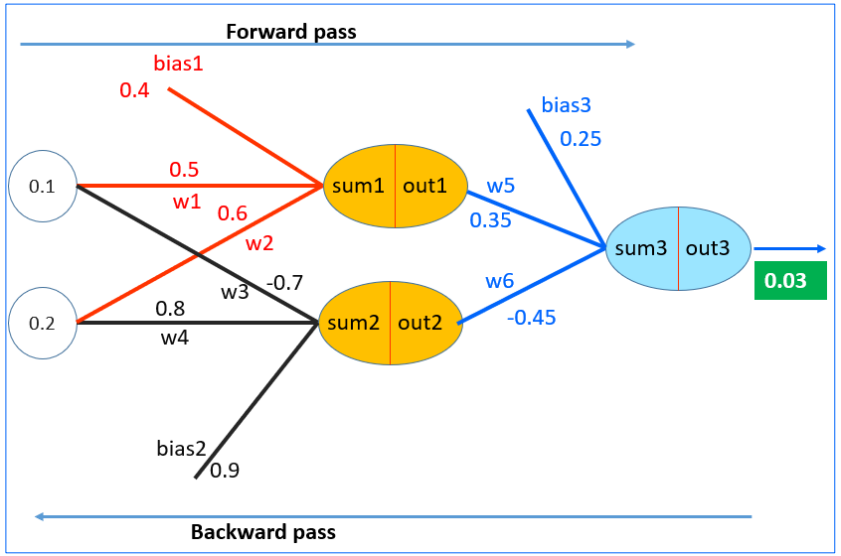
sum1 =w1*x1+w2*x2+bias1

d_sum1_bias1 =1

dloss_bias1=(predicted_y-y)* sigmoid_derivative(predicted_y)* w5* sigmoid_derivative(out1)* 1

Backpropagation – MultiLayer NN

Utilisation de chaine rule pour trouver $\frac{\partial Loss}{\partial bias_hidden}$



$$\frac{\partial Loss}{\partial bias2} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial out2} * \frac{\partial out2}{\partial sum2} * \frac{\partial sum2}{\partial bias2}$$

$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

dsum3_dout2= w6

out2 =sigmoid(sum2)

d_out1_dsum2=sigmoid_derivative(out2)

sum2 =w3*x1+w4*x2+bias2

d_sum2_bias2 =1

dloss_bias2=(predicted_y-y)* sigmoid_derivative(predicted_y)* w6* sigmoid_derivative(out2)* 1

Update hidden layer bias

$$\frac{\partial Loss}{\partial bias_hidden}$$

```
dloss_bias1=(predicted_y-y)* sigmoid_derivative(predicted_y)* w5*
sigmoid_derivative(out1)* 1
dloss_bias2=(predicted_y-y)* sigmoid_derivative(predicted_y)* w6*
sigmoid_derivative(out2)* 1
```

```
dloss_bias_h=(predicted_y-y)* sigmoid_derivative(predicted_y)* output_W *
sigmoid_derivative(h_out)
```

Code python

```
dloss_dbias_h=(predicted_y-y)*
sigmoid_derivative(predicted_y).dot(output_weights.T)*sigmoid_derivative(hidden_layer_out)
```

2) Update output layer weights and bias

Update output layer weights and bias

$$\frac{\partial Loss}{\partial Wout}$$

Calculer $w_{out} = w_{out} - learning_rate * \frac{\partial loss}{\partial w_{out}}$

$$\frac{\partial Loss}{\partial bias_out}$$

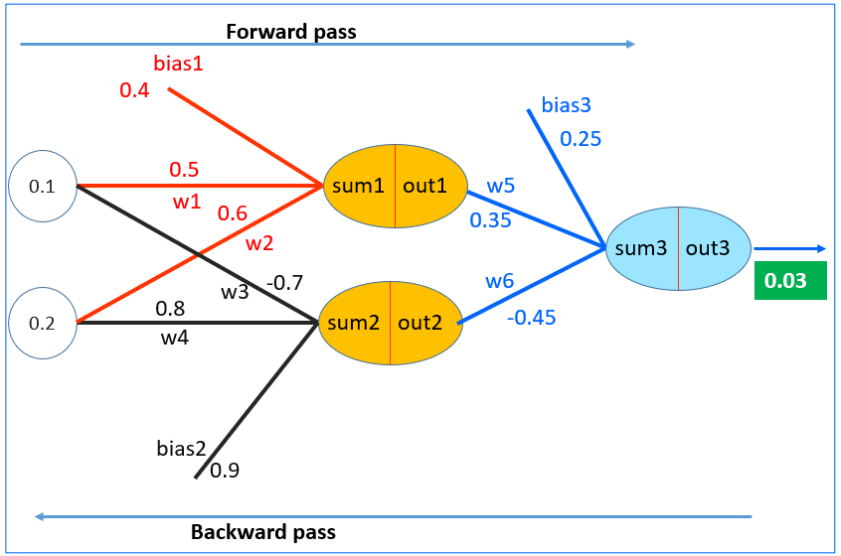
Calculer $bias_{out} = bias_{out} - learning_rate * \frac{\partial loss}{\partial w_{bias}}$

Update output layer weights

$$\frac{\partial Loss}{\partial Wout}$$

Backpropagation – MultiLayer NN

Utilisation de chaine rule pour trouver $\frac{\partial Loss}{\partial Wout}$



$$\frac{\partial Loss}{\partial w5} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial W5}$$

$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

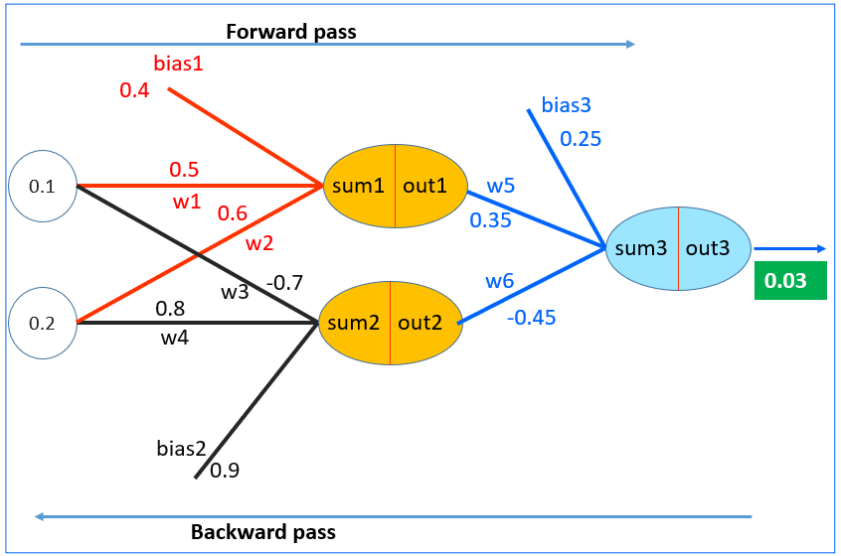
d_sum3_dw5 =out1

dloss_dw5=**(predicted_y-y)* sigmoid_derivative(predicted_y) * out1**

Backpropagation – MultiLayer NN

Utilisation de chaine rule pour trouver $\frac{\partial Loss}{\partial Wout}$

$$\frac{\partial Loss}{\partial w6} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial W6}$$



$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

d_sum3_dw6 =out2

dloss_dw6=(predicted_y-y)* sigmoid_derivative(predicted_y) * out2

$$\frac{\partial Loss}{\partial Wout}$$

```
dloss_dw5=(predicted_y-y)* sigmoid_derivative(predicted_y) * out1
```

```
dloss_dw6=(predicted_y-y)* sigmoid_derivative(predicted_y) * out2
```

```
dloss_dw_out=(predicted_y-y)* sigmoid_derivative(predicted_y) * h_out
```

Code python

```
dloss_dw_out=hidden_layer_out.T.dot((predicted_y-y)*sigmoid_derivative(predicted_y))
```

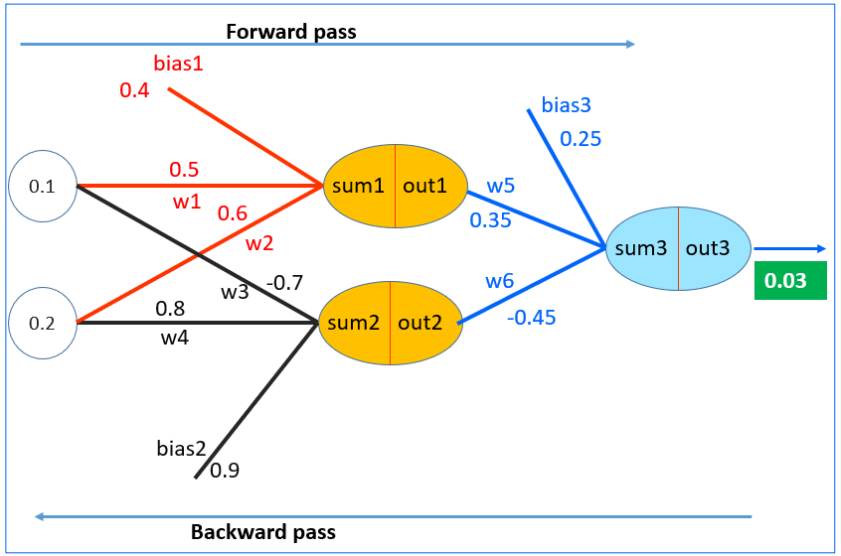
Update output layer bias

$$\frac{\partial Loss}{\partial bias_o}$$

Backpropagation – MultiLayer NN

Utilisation de chaine rule pour trouver $\frac{\partial Loss}{\partial Bias_out}$

$$\frac{\partial Loss}{\partial bias3} = \frac{\partial Loss}{\partial predicted_y} * \frac{\partial predicted_y}{\partial sum3} * \frac{\partial sum3}{\partial bias3}$$



$$Loss = \frac{1}{2}(y - predicted_y)^2$$

dloss_depredicted=(predicted_y-y)

predicted_y =sigmoid(sum3)

dpredecited_dsum3= sigmoid_derivative(predicted_y)

sum3 =w5*out1+w6*out2+bias3

d_sum3_dbias3 =1

dloss_bias3=(predicted_y-y)* sigmoid_derivative(predicted_y) * 1

$$\frac{\partial Loss}{\partial bias_o}$$

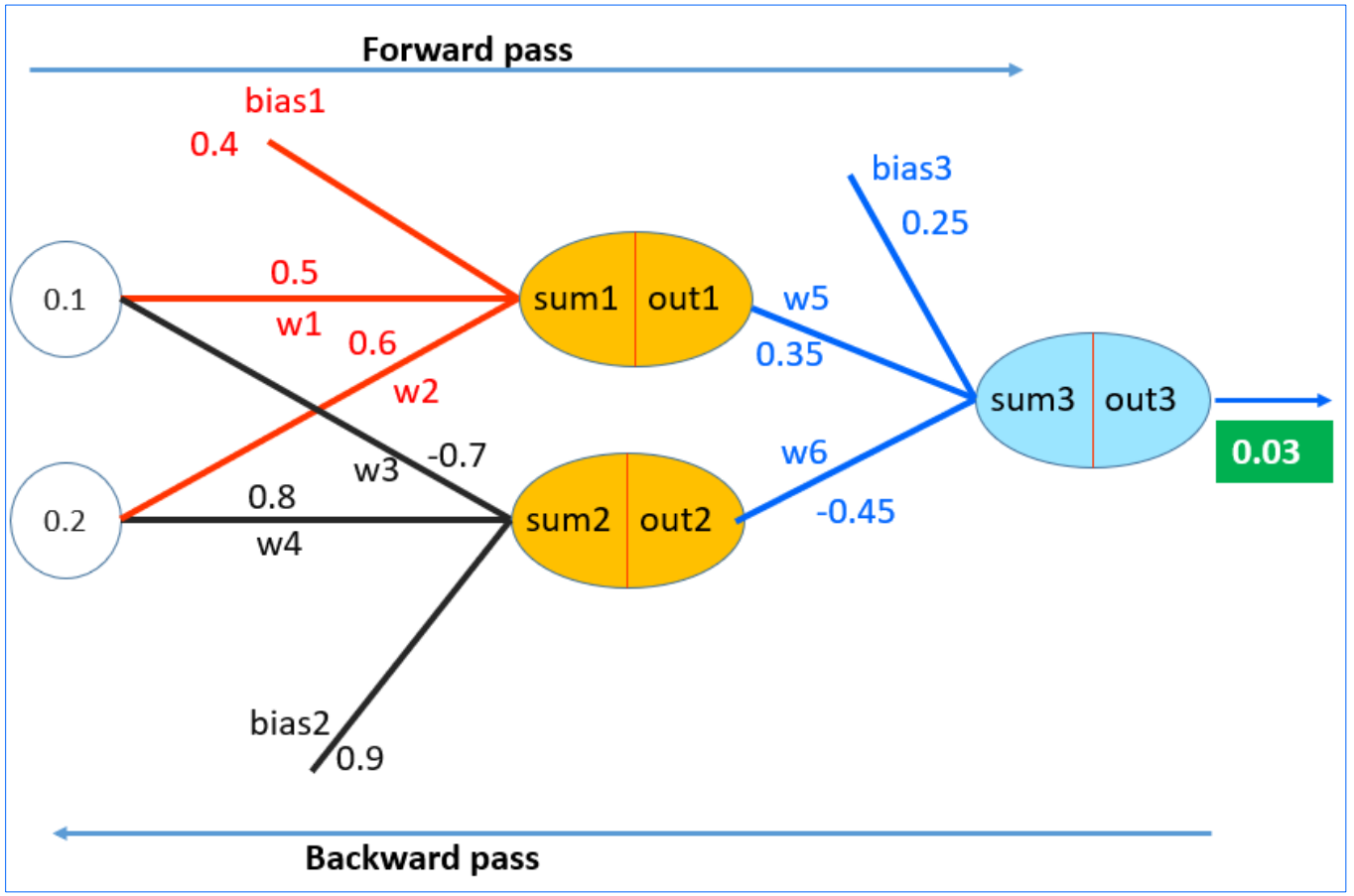
```
dloss_bias3=(predicted_y-y)* sigmoid_derivative(predicted_y) * 1
```

```
dloss_dbias_out=(predicted_y-y)* sigmoid_derivative(predicted_y)
```

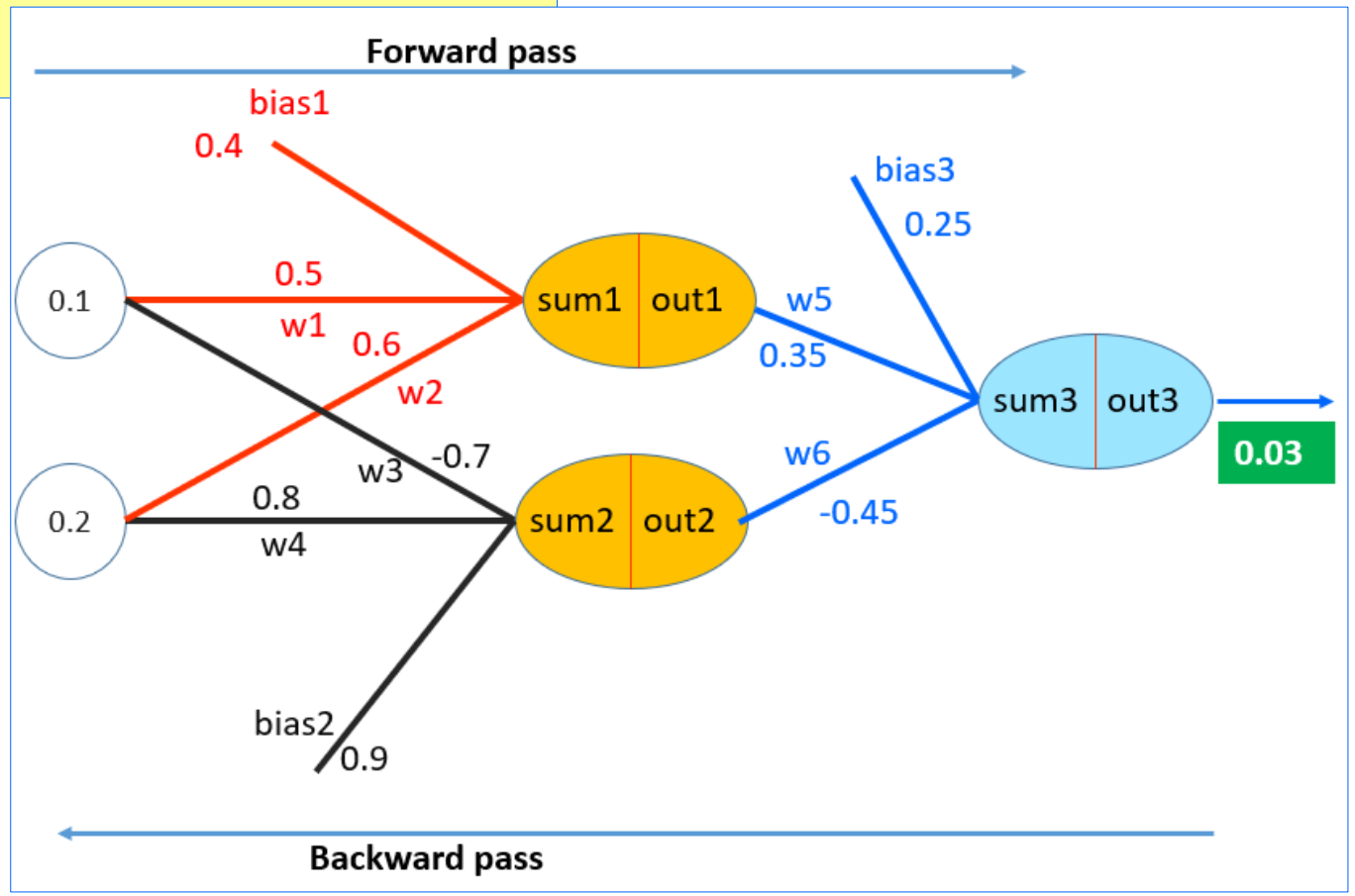
Code python

```
dloss_dbias_out = (predicted_y-y)*sigmoid_derivative(predicted_y)
```

```
#inputs
X = np.array([[0.1,0.2]])
y = np.array([[0.03]])
learning_rate= 0.1
#weights and bias initialization
hidden_weights = [[0.5,0.6],[-0.7,0.8]]
hidden_bias =[0.4,0.9]
output_weights = np.array([[0.35],[-0.45]])
output_bias = np.array([[0.25]])
```



```
#Learning process

#Forward Pass

hidden_layer_sum = np.dot(X,hidden_weights) + hidden_bias

hidden_layer_out = sigmoid(hidden_layer_sum)

output_layer_sum = np.dot(hidden_layer_out,output_weights) +
output_bias

predicted_y = sigmoid(output_layer_sum)
```



```
#Learning process

#Backward pass

#hidden layer

dloss_dw_h= X.T.dot((predicted_y-y)*
sigmoid_derivative(predicted_y).dot(output_weights.T)*sigmoid_derivative(hidden_layer_out))

dloss_dbias_h=(predicted_y-y)*
sigmoid_derivative(predicted_y).dot(output_weights.T)*sigmoid_derivative(hidden_layer_out)

#output layer

dloss_dw_out=hidden_layer_out.T.dot((predicted_y-y)*sigmoid_derivative(predicted_y))

dloss_dbias_out=(predicted_y-y)*sigmoid_derivative(predicted_y)
```

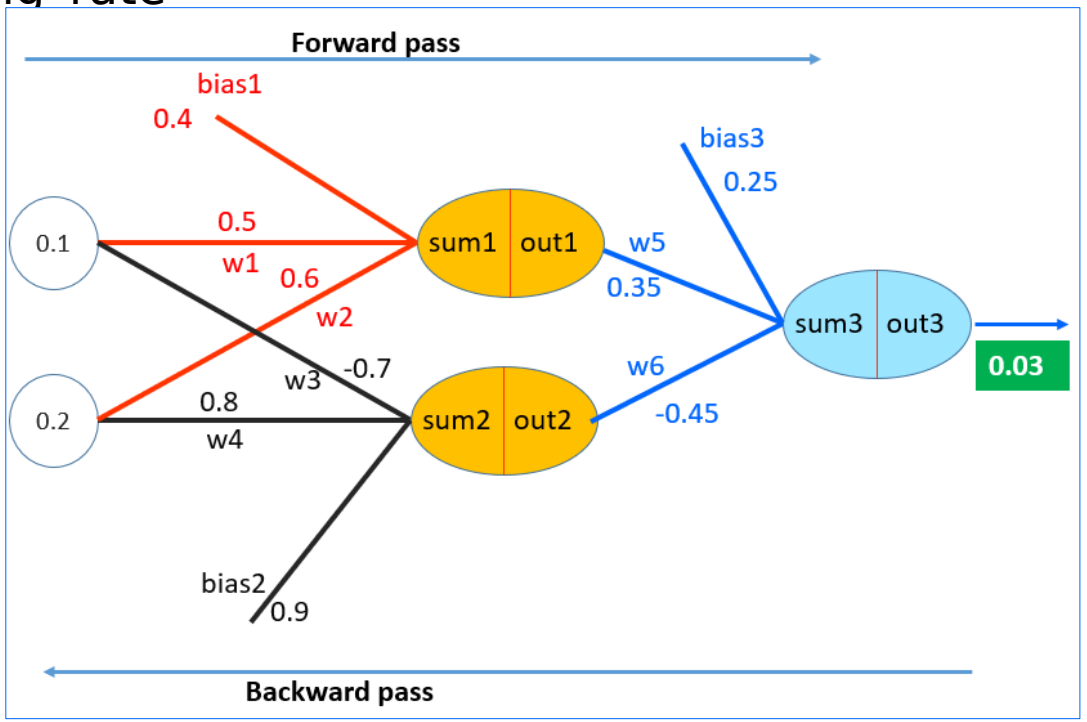
```
dloss_dbias_h=(predicted_y-y)*
sigmoid_derivative(predicted_y).dot(output_weights.T)*sigmoid_derivative(hidden_layer_out)
```


```
dloss_dbias_h=(predicted_y-y)*
sigmoid_derivative(predicted_y).dot(output_weights.T)*sigmoid_derivative(hidden_layer_out)
```


```
dloss_dw_out=hidden_layer_out.T.dot((predicted_y-y)*sigmoid_derivative(predicted_y))
```

```
dloss_dbias_out = (predicted_y-y)*sigmoid_derivative(predicted_y)
```

```
#update
hidden_weights-=dloss_dw_h*learning_rate
hidden_bias-=dloss_dbias_h*learning_rate
hidden_bias=np.sum(hidden_bias,axis=0,keepdims=True)
output_weights-=dloss_dw_out*learning_rate
output_bias-
=np.sum(dloss_dbias_out,axis=0,keepdims=True)*learning_rate
```



```
#print results
print("results-----------------------------------")
print("updated hidden weights: ")
print(hidden_weights)
print("updated hidden biases: ")
print(hidden_bias)
print("updated output weights: ")
print(output_weights)
print("updated output biases: ")
print(output_bias)
```